- How to achieve those qualities
- How to determine the design decisions we might make with respect to those qualities

This chapter provides the context for the discussion of specific quality attributes in Chapters 5–11.

4.1 Architecture and Requirements

Requirements for a system come in a variety of forms: textual requirements, mockups, existing systems, use cases, user stories, and more. Chapter 16 discusses the concept of an *architecturally significant requirement*, the role such requirements play in architecture, and how to identify them. No matter the source, all requirements encompass the following categories:

1. *Functional requirements.* These requirements state what the system must do, and how it must behave or react to runtime stimuli.
2. *Quality attribute requirements.* These requirements are qualifications of the functional requirements or of the overall product. A qualification of a functional requirement is an item such as how fast the function must be performed, or how resilient it must be to erroneous input. A qualification of the overall product is an item such as the time to deploy the product or a limitation on operational costs.
3. *Constraints.* A constraint is a design decision with zero degrees of freedom. That is, it's a design decision that's already been made. Examples include the requirement to use a certain programming language or to reuse a certain existing module, or a management fiat to make your system service oriented. These choices are arguably in the purview of the architect, but external factors (such as not being able to train the staff in a new language, or having a business agreement with a software supplier, or pushing business goals of service interoperability) have led those in power to dictate these design outcomes.

What is the "response" of architecture to each of these kinds of requirements?

1. Functional requirements are satisfied by assigning an appropriate sequence of responsibilities throughout the design. As we will see later in this chapter, assigning responsibilities to architectural elements is a fundamental architectural design decision.
2. Quality attribute requirements are satisfied by the various structures designed into the architecture, and the behaviors and interactions of the elements that populate those structures. Chapter 17 will show this approach in more detail.

3. Constraints are satisfied by accepting the design decision and reconciling it with other affected design decisions.

4.2 Functionality

Functionality is the ability of the system to do the work for which it was intended. Of all of the requirements, functionality has the strangest relationship to architecture.

First of all, functionality does not determine architecture. That is, given a set of required functionality, there is no end to the architectures you could create to satisfy that functionality. At the very least, you could divide up the functionality in any number of ways and assign the subpieces to different architectural elements.

In fact, if functionality were the only thing that mattered, you wouldn't have to divide the system into pieces at all; a single monolithic blob with no internal structure would do just fine. Instead, we design our systems as structured sets of cooperating architectural elements—modules, layers, classes, services, databases, apps, threads, peers, tiers, and on and on—to make them understandable and to support a variety of other purposes. Those "other purposes" are the other quality attributes that we'll turn our attention to in the remaining sections of this chapter, and the remaining chapters of Part II.

But although functionality is independent of any particular structure, functionality is achieved by assigning responsibilities to architectural elements, resulting in one of the most basic of architectural structures.

Although responsibilities can be allocated arbitrarily to any modules, software architecture constrains this allocation when other quality attributes are important. For example, systems are frequently divided so that several people can cooperatively build them. The architect's interest in functionality is in how it interacts with and constrains other qualities.

4.3 Quality Attribute Considerations

Just as a system's functions do not stand on their own without due consideration of other quality attributes, neither do quality attributes stand on their own; they pertain to the functions of the system. If a functional requirement is "When the user presses the green button, the Options dialog appears," a performance QA annotation might describe how quickly the dialog will appear; an availability QA annotation might describe how often this function will fail, and how quickly it will be repaired; a usability QA annotation might describe how easy it is to learn this function.

Functional Requirements

After more than 15 years of writing and discussing the distinction between functional requirements and quality requirements, the definition of functional requirements still eludes me. Quality attribute requirements are well defined: performance has to do with the timing behavior of the system, modifiability has to do with the ability of the system to support changes in its behavior or other qualities after initial deployment, availability has to do with the ability of the system to survive failures, and so forth.

Function, however, is much more slippery. An international standard (ISO 25010) defines functional suitability as "the capability of the software product to provide functions which meet stated and implied needs when the software is used under specified conditions." That is, functionality is the ability to provide functions. One interpretation of this definition is that functionality describes what the system does and quality describes how well the system does its function. That is, qualities are attributes of the system and function is the purpose of the system.

This distinction breaks down, however, when you consider the nature of some of the "function." If the function of the software is to control engine behavior, how can the function be correctly implemented without considering timing behavior? Is the ability to control access through requiring a user name/password combination not a function even though it is not the purpose of any system?

I like much better the use of the word "responsibility" to describe computations that a system must perform. Questions such as "What are the timing constraints on that set of responsibilities?", "What modifications are anticipated with respect to that set of responsibilities?", and "What class of users is allowed to execute that set of responsibilities?" make sense and are actionable.

The achievement of qualities induces responsibility; think of the user name/password example just mentioned. Further, one can identify responsibilities as being associated with a particular set of requirements.

So does this mean that the term "functional requirement" shouldn't be used? People have an understanding of the term, but when precision is desired, we should talk about sets of specific responsibilities instead.

Paul Clements has long ranted against the careless use of the term "nonfunctional," and now it's my turn to rant against the careless use of the term "functional"—probably equally ineffectually.

—LB

Quality attributes have been of interest to the software community at least since the 1970s. There are a variety of published taxonomies and definitions, and many of them have their own research and practitioner communities. From an

architect's perspective, there are three problems with previous discussions of system quality attributes:

1. The definitions provided for an attribute are not testable. It is meaningless to say that a system will be "modifiable." Every system may be modifiable with respect to one set of changes and not modifiable with respect to another. The other quality attributes are similar in this regard: a system may be robust with respect to some faults and brittle with respect to others. And so forth.
2. Discussion often focuses on which quality a particular concern belongs to. Is a system failure due to a denial-of-service attack an aspect of availability, an aspect of performance, an aspect of security, or an aspect of usability? All four attribute communities would claim ownership of a system failure due to a denial-of-service attack. All are, to some extent, correct. But this doesn't help us, as architects, understand and create architectural solutions to manage the attributes of concern.
3. Each attribute community has developed its own vocabulary. The performance community has "events" arriving at a system, the security community has "attacks" arriving at a system, the availability community has "failures" of a system, and the usability community has "user input." All of these may actually refer to the same occurrence, but they are described using different terms.

A solution to the first two of these problems (untestable definitions and overlapping concerns) is to use *quality attribute scenarios* as a means of characterizing quality attributes (see the next section). A solution to the third problem is to provide a discussion of each attribute—concentrating on its underlying concerns—to illustrate the concepts that are fundamental to that attribute community.

There are two categories of quality attributes on which we focus. The first is those that describe some property of the system at runtime, such as availability, performance, or usability. The second is those that describe some property of the development of the system, such as modifiability or testability.

Within complex systems, quality attributes can never be achieved in isolation. The achievement of any one will have an effect, sometimes positive and sometimes negative, on the achievement of others. For example, almost every quality attribute negatively affects performance. Take portability. The main technique for achieving portable software is to isolate system dependencies, which introduces overhead into the system's execution, typically as process or procedure boundaries, and this hurts performance. Determining the design that satisfies all of the quality attribute requirements is partially a matter of making the appropriate tradeoffs; we discuss design in Chapter 17. Our purpose here is to provide the context for discussing each quality attribute. In particular, we focus on how quality attributes can be specified, what architectural decisions will enable the achievement of particular quality attributes, and what questions about quality attributes will enable the architect to make the correct design decisions.

4.4 Specifying Quality Attribute Requirements

A quality attribute requirement should be unambiguous and testable. We use a common form to specify all quality attribute requirements. This has the advantage of emphasizing the commonalities among all quality attributes. It has the disadvantage of occasionally being a force-fit for some aspects of quality attributes.

Our common form for quality attribute expression has these parts:

- *Stimulus.* We use the term "stimulus" to describe an event arriving at the system. The stimulus can be an event to the performance community, a user operation to the usability community, or an attack to the security community. We use the same term to describe a motivating action for developmental qualities. Thus, a stimulus for modifiability is a request for a modification; a stimulus for testability is the completion of a phase of development.
- *Stimulus source.* A stimulus must have a source—it must come from somewhere. The source of the stimulus may affect how it is treated by the system. A request from a trusted user will not undergo the same scrutiny as a request by an untrusted user.
- *Response.* How the system should respond to the stimulus must also be specified. The response consists of the responsibilities that the system (for runtime qualities) or the developers (for development-time qualities) should perform in response to the stimulus. For example, in a performance scenario, an event arrives (the stimulus) and the system should process that event and generate a response. In a modifiability scenario, a request for a modification arrives (the stimulus) and the developers should implement the modification—without side effects—and then test and deploy the modification.
- *Response measure.* Determining whether a response is satisfactory—whether the requirement is satisfied—is enabled by providing a response measure. For performance this could be a measure of latency or throughput; for modifiability it could be the labor or wall clock time required to make, test, and deploy the modification.

These four characteristics of a scenario are the heart of our quality attribute specifications. But there are two more characteristics that are important: environment and artifact.

- *Environment.* The environment of a requirement is the set of circumstances in which the scenario takes place. The environment acts as a qualifier on the stimulus. For example, a request for a modification that arrives after the code has been frozen for a release may be treated differently than one that arrives before the freeze. A failure that is the fifth successive failure

of a component may be treated differently than the first failure of that component.

- *Artifact.* Finally, the artifact is the portion of the system to which the requirement applies. Frequently this is the entire system, but occasionally specific portions of the system may be called out. A failure in a data store may be treated differently than a failure in the metadata store. Modifications to the user interface may have faster response times than modifications to the middleware.

To summarize how we specify quality attribute requirements, we capture them formally as six-part scenarios. While it is common to omit one or more of these six parts, particularly in the early stages of thinking about quality attributes, knowing that all parts are there forces the architect to consider whether each part is relevant.

In summary, here are the six parts:

1. *Source of stimulus.* This is some entity (a human, a computer system, or any other actuator) that generated the stimulus.
2. *Stimulus.* The stimulus is a condition that requires a response when it arrives at a system.
3. *Environment.* The stimulus occurs under certain conditions. The system may be in an overload condition or in normal operation, or some other relevant state. For many systems, "normal" operation can refer to one of a number of modes. For these kinds of systems, the environment should specify in which mode the system is executing.
4. *Artifact.* Some artifact is stimulated. This may be a collection of systems, the whole system, or some piece or pieces of it.
5. *Response.* The response is the activity undertaken as the result of the arrival of the stimulus.
6. *Response measure.* When the response occurs, it should be measurable in some fashion so that the requirement can be tested.

We distinguish *general* quality attribute scenarios (which we call "general scenarios" for short)—those that are system independent and can, potentially, pertain to any system—from *concrete* quality attribute scenarios (concrete scenarios)—those that are specific to the particular system under consideration.

We can characterize quality attributes as a collection of general scenarios. Of course, to translate these generic attribute characterizations into requirements for a particular system, the general scenarios need to be made system specific. Detailed examples of these scenarios will be given in Chapters 5–11. Figure 4.1 shows the parts of a quality attribute scenario that we have just discussed. Figure 4.2 shows an example of a general scenario, in this case for availability.

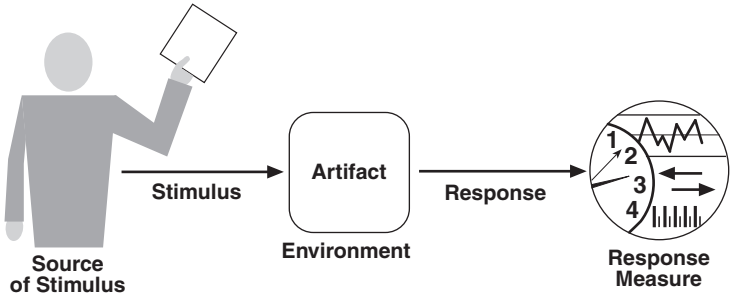


FIGURE 4.1 The parts of a quality attribute scenario

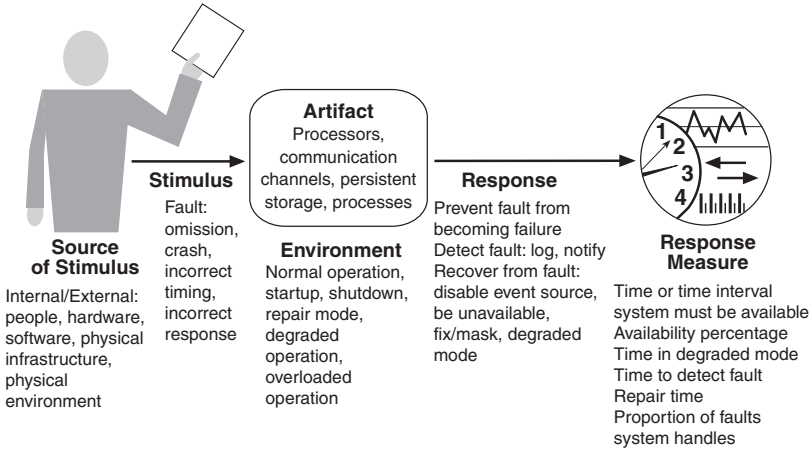


FIGURE 4.2 A general scenario for availability

4.5 Achieving Quality Attributes through Tactics

The quality attribute requirements specify the responses of the system that, with a bit of luck and a dose of good planning, realize the goals of the business. We now turn to the techniques an architect can use to *achieve* the required quality attributes. We call these techniques *architectural tactics.* A tactic is a design decision that influences the achievement of a quality attribute response—tactics directly affect the system's response to some stimulus. Tactics impart portability to one design, high performance to another, and integrability to a third.

Not My Problem

One time I was doing an architecture analysis on a complex system created by and for Lawrence Livermore National Laboratory. If you visit their website (www.llnl.gov) and try to figure out what Livermore Labs does, you will see the word "security" mentioned over and over. The lab focuses on nuclear security, international and domestic security, and environmental and energy security. Serious stuff . . .

Keeping this emphasis in mind, I asked them to describe the quality attributes of concern for the system that I was analyzing. I'm sure you can imagine my surprise when security wasn't mentioned once! The system stakeholders mentioned performance, modifiability, evolvability, interoperability, configurability, and portability, and one or two more, but the word security never passed their lips.

Being a good analyst, I questioned this seemingly shocking and obvious omission. Their answer was simple and, in retrospect, straightforward: "We don't care about it. Our systems are not connected to any external network and we have barbed-wire fences and guards with machine guns." Of course, *someone* at Livermore Labs was very interested in security. But it was clearly not the software architects.

—RK

The focus of a tactic is on a single quality attribute response. Within a tactic, there is no consideration of tradeoffs. Tradeoffs must be explicitly considered and controlled by the designer. In this respect, tactics differ from architectural patterns, where tradeoffs are built into the pattern. (We visit the relation between tactics and patterns in Chapter 14. Chapter 13 explains how sets of tactics for a quality attribute can be constructed, which are the steps we used to produce the set in this book.)

A system design consists of a collection of decisions. Some of these decisions help control the quality attribute responses; others ensure achievement of system functionality. We represent the relationship between stimulus, tactics, and response in Figure 4.3. The tactics, like design patterns, are design techniques that architects have been using for years. Our contribution is to isolate, catalog, and describe them. We are not inventing tactics here, we are just capturing what architects do in practice.

Why do we do this? There are three reasons:

1. Design patterns are complex; they typically consist of a bundle of design decisions. But patterns are often difficult to apply as is; architects need to modify and adapt them. By understanding the role of tactics, an architect can more easily assess the options for augmenting an existing pattern to achieve a quality attribute goal.

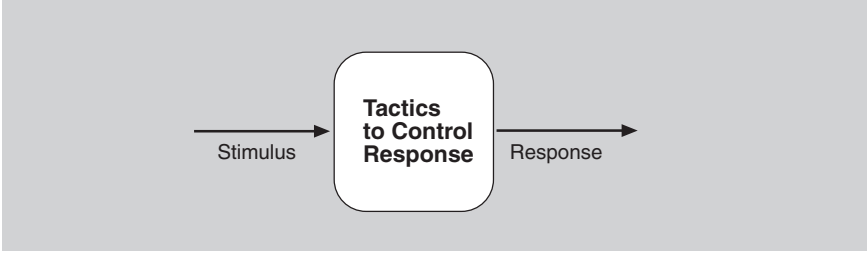


FIGURE 4.3 Tactics are intended to control responses to stimuli.

2. If no pattern exists to realize the architect's design goal, tactics allow the architect to construct a design fragment from "first principles." Tactics give the architect insight into the properties of the resulting design fragment.
3. By cataloging tactics, we provide a way of making design more systematic within some limitations. Our list of tactics does not provide a taxonomy. We only provide a categorization. The tactics will overlap, and you frequently will have a choice among multiple tactics to improve a particular quality attribute. The choice of which tactic to use depends on factors such as tradeoffs among other quality attributes and the cost to implement. These considerations transcend the discussion of tactics for particular quality attributes. Chapter 17 provides some techniques for choosing among competing tactics.

The tactics that we present can and should be refined. Consider performance: *Schedule resources* is a common performance tactic. But this tactic needs to be refined into a specific scheduling strategy, such as shortest-job-first, round-robin, and so forth, for specific purposes. *Use an intermediary* is a modifiability tactic. But there are multiple types of intermediaries (layers, brokers, and proxies, to name just a few). Thus there are refinements that a designer will employ to make each tactic concrete.

In addition, the application of a tactic depends on the context. Again considering performance: *Manage sampling rate* is relevant in some real-time systems but not in all real-time systems and certainly not in database systems.

4.6 Guiding Quality Design Decisions

Recall that one can view an architecture as the result of applying a collection of design decisions. What we present here is a systematic categorization of these

decisions so that an architect can focus attention on those design dimensions likely to be most troublesome.

The seven categories of design decisions are

1. Allocation of responsibilities
2. Coordination model
3. Data model
4. Management of resources
5. Mapping among architectural elements
6. Binding time decisions
7. Choice of technology

These categories are not the only way to classify architectural design decisions, but they do provide a rational division of concerns. These categories might overlap, but it's all right if a particular decision exists in two different categories, because the concern of the architect is to ensure that every important decision is considered. Our categorization of decisions is partially based on our definition of software architecture in that many of our categories relate to the definition of structures and the relations among them.

Allocation of Responsibilities

Decisions involving allocation of responsibilities include the following:

- Identifying the important responsibilities, including basic system functions, architectural infrastructure, and satisfaction of quality attributes.
- Determining how these responsibilities are allocated to non-runtime and runtime elements (namely, modules, components, and connectors).

Strategies for making these decisions include functional decomposition, modeling real-world objects, grouping based on the major modes of system operation, or grouping based on similar quality requirements: processing frame rate, security level, or expected changes.

In Chapters 5–11, where we apply these design decision categories to a number of important quality attributes, the checklists we provide for the allocation of responsibilities category is derived systematically from understanding the stimuli and responses listed in the general scenario for that QA.

Coordination Model

Software works by having elements interact with each other through designed mechanisms. These mechanisms are collectively referred to as a coordination model. Decisions about the coordination model include these:

- Identifying the elements of the system that must coordinate, or are prohibited from coordinating.
- Determining the properties of the coordination, such as timeliness, currency, completeness, correctness, and consistency.
- Choosing the communication mechanisms (between systems, between our system and external entities, between elements of our system) that realize those properties. Important properties of the communication mechanisms include stateful versus stateless, synchronous versus asynchronous, guaranteed versus nonguaranteed delivery, and performance-related properties such as throughput and latency.

Data Model

Every system must represent artifacts of system-wide interest—data—in some internal fashion. The collection of those representations and how to interpret them is referred to as the data model. Decisions about the data model include the following:

- Choosing the major data abstractions, their operations, and their properties. This includes determining how the data items are created, initialized, accessed, persisted, manipulated, translated, and destroyed.
- Compiling metadata needed for consistent interpretation of the data.
- Organizing the data. This includes determining whether the data is going to be kept in a relational database, a collection of objects, or both. If both, then the mapping between the two different locations of the data must be determined.

Management of Resources

An architect may need to arbitrate the use of shared resources in the architecture. These include hard resources (e.g., CPU, memory, battery, hardware buffers, system clock, I/O ports) and soft resources (e.g., system locks, software buffers, thread pools, and non-thread-safe code).

Decisions for management of resources include the following:

- Identifying the resources that must be managed and determining the limits for each.
- Determining which system element(s) manage each resource.
- Determining how resources are shared and the arbitration strategies employed when there is contention.
- Determining the impact of saturation on different resources. For example, as a CPU becomes more heavily loaded, performance usually just degrades fairly steadily. On the other hand, when you start to run out of memory, at

some point you start paging/swapping intensively and your performance suddenly crashes to a halt.

Mapping among Architectural Elements

An architecture must provide two types of mappings. First, there is mapping between elements in different types of architecture structures—for example, mapping from units of development (modules) to units of execution (threads or processes). Next, there is mapping between software elements and environment elements—for example, mapping from processes to the specific CPUs where these processes will execute.

Useful mappings include these:

- The mapping of modules and runtime elements to each other—that is, the runtime elements that are created from each module; the modules that contain the code for each runtime element.
- The assignment of runtime elements to processors.
- The assignment of items in the data model to data stores.
- The mapping of modules and runtime elements to units of delivery.

Binding Time Decisions

Binding time decisions introduce allowable ranges of variation. This variation can be bound at different times in the software life cycle by different entities—from design time by a developer to runtime by an end user. A binding time decision establishes the scope, the point in the life cycle, and the mechanism for achieving the variation.

The decisions in the other six categories have an associated binding time decision. Examples of such binding time decisions include the following:

- For allocation of responsibilities, you can have build-time selection of modules via a parameterized makefile.
- For choice of coordination model, you can design runtime negotiation of protocols.
- For resource management, you can design a system to accept new peripheral devices plugged in at runtime, after which the system recognizes them and downloads and installs the right drivers automatically.
- For choice of technology, you can build an app store for a smartphone that automatically downloads the version of the app appropriate for the phone of the customer buying the app.

When making binding time decisions, you should consider the costs to implement the decision and the costs to make a modification after you have implemented the decision. For example, if you are considering changing platforms

at some time after code time, you can insulate yourself from the effects caused by porting your system to another platform at some cost. Making this decision depends on the costs incurred by having to modify an early binding compared to the costs incurred by implementing the mechanisms involved in the late binding.

Choice of Technology

Every architecture decision must eventually be realized using a specific technology. Sometimes the technology selection is made by others, before the intentional architecture design process begins. In this case, the chosen technology becomes a constraint on decisions in each of our seven categories. In other cases, the architect must choose a suitable technology to realize a decision in every one of the categories.

Choice of technology decisions involve the following:

- Deciding which technologies are available to realize the decisions made in the other categories.
- Determining whether the available tools to support this technology choice (IDEs, simulators, testing tools, etc.) are adequate for development to proceed.
- Determining the extent of internal familiarity as well as the degree of external support available for the technology (such as courses, tutorials, examples, and availability of contractors who can provide expertise in a crunch) and deciding whether this is adequate to proceed.
- Determining the side effects of choosing a technology, such as a required coordination model or constrained resource management opportunities.
- Determining whether a new technology is compatible with the existing technology stack. For example, can the new technology run on top of or alongside the existing technology stack? Can it communicate with the existing technology stack? Can the new technology be monitored and managed?

4.7 Summary

Requirements for a system come in three categories:

1. *Functional.* These requirements are satisfied by including an appropriate set of responsibilities within the design.
2. *Quality attribute.* These requirements are satisfied by the structures and behaviors of the architecture.
3. *Constraints.* A constraint is a design decision that's already been made.

To express a quality attribute requirement, we use a quality attribute scenario. The parts of the scenario are these:

1. Source of stimulus
2. Stimulus
3. Environment
4. Artifact
5. Response
6. Response measure

An architectural tactic is a design decision that affects a quality attribute response. The focus of a tactic is on a single quality attribute response. Architectural patterns can be seen as "packages" of tactics.

The seven categories of architectural design decisions are these:

1. Allocation of responsibilities
2. Coordination model
3. Data model
4. Management of resources
5. Mapping among architectural elements
6. Binding time decisions
7. Choice of technology

4.8 For Further Reading

Philippe Kruchten [Kruchten 04] provides another categorization of design decisions.

Pena [Pena 87] uses categories of Function/Form/Economy/Time as a way of categorizing design decisions.

Binding time and mechanisms to achieve different types of binding times are discussed in [Bachmann 05].

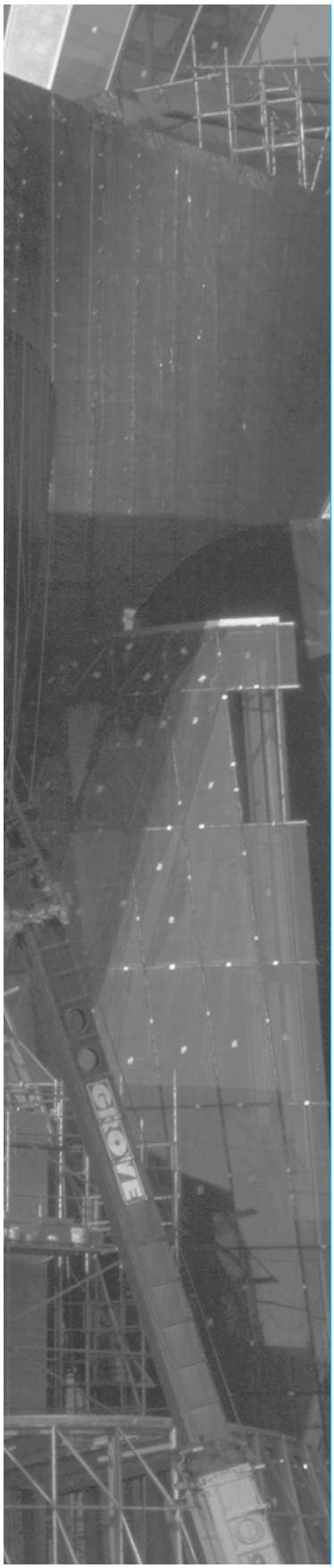
Taxonomies of quality attributes can be found in [Boehm 78], [McCall 77], and [ISO 11].

Arguments for viewing architecture as essentially independent from function can be found in [Shaw 95].

4.9 Discussion Questions

1. What is the relationship between a use case and a quality attribute scenario? If you wanted to add quality attribute information to a use case, how would you do it?

2. Do you suppose that the set of tactics for a quality attribute is finite or infinite? Why?
3. Discuss the choice of programming language (an example of choice of technology) and its relation to architecture in general, and the design decisions in the other six categories? For instance, how can certain programming languages enable or inhibit the choice of particular coordination models?
4. We will be using the automatic teller machine as an example throughout the chapters on quality attributes. Enumerate the set of responsibilities that an automatic teller machine should support and propose an initial design to accommodate that set of responsibilities. Justify your proposal.
5. Think about the screens that your favorite automatic teller machine uses. What do those screens tell you about binding time decisions reflected in the architecture?
6. Consider the choice between synchronous and asynchronous communication (a choice in the coordination mechanism category). What quality attribute requirements might lead you to choose one over the other?
7. Consider the choice between stateful and stateless communication (a choice in the coordination mechanism category). What quality attribute requirements might lead you to choose one over the other?
8. Most peer-to-peer architecture employs late binding of the topology. What quality attributes does this promote or inhibit?



12

Dependability and security specification

Objectives

The objective of this chapter is to explain how to specify functional and non-functional dependability and security requirements. When you have read this chapter, you will:

- understand how a risk-driven approach can be used for identifying and analyzing safety, reliability, and security requirements;
- understand how fault trees can be used to help analyze risks and derive safety requirements;
- have been introduced to metrics for reliability specification and how these are used to specify measurable reliability requirements;
- know the different types of security requirements that may be required in a complex system;
- be aware of the advantages and disadvantages of using formal, mathematical specifications of a system.

Contents

12.1 Risk-driven requirements specification
12.2 Safety specification
12.3 Reliability specification
12.4 Security specification
12.5 Formal specification

SR1: The system shall not deliver a single dose of insulin that is greater than a specified maximum dose for a system user.

SR2: The system shall not deliver a daily cumulative dose of insulin that is greater than a specified maximum daily dose for a system user.

SR3: The system shall include a hardware diagnostic facility that shall be executed at least four times per hour.

SR4: The system shall include an exception handler for all of the exceptions that are identified in Table 3.

SR5: The audible alarm shall be sounded when any hardware or software anomaly is discovered and a diagnostic message, as defined in Table 4, shall be displayed.

SR6: In the event of an alarm, insulin delivery shall be suspended until the user has reset the system and cleared the alarm.

Figure 12.5 Examples of safety requirements

software failures that could lead to an incorrect dose of insulin are considered, the following 'solutions' might be developed:

1. *Arithmetic error* This may occur when an arithmetic computation causes a representation failure. The specification should identify all possible arithmetic errors that may occur and state that an exception handler must be included for each possible error. The specification should set out the action to be taken for each of these errors. The default safe action is to shut down the delivery system and activate a warning alarm.

2. *Algorithmic error* This is a more difficult situation as there is no clear program exception that must be handled. This type of error could be detected by comparing the required insulin dose computed with the previously delivered dose. If it is much higher, this may mean that the amount has been computed incorrectly. The system may also keep track of the dose sequence. After a number of above-average doses have been delivered, a warning may be issued and further dosage limited.

Some of the resulting safety requirements for the insulin pump software are shown in Figure 12.5. These are user requirements and, naturally, they would be expressed in more detail in the system requirements specification. In Figure 12.5, the references to Tables 3 and 4 relate to tables that are included in the requirements document—they are not shown here.

12.3 Reliability specification

As I discussed in Chapter 10, the overall reliability of a system depends on the hardware reliability, the software reliability, and the reliability of the system operators. The system software has to take this into account. As well as including requirements

that compensate for software failure, there may also be related reliability requirements to help detect and recover from hardware failures and operator errors.

Reliability is different from safety and security in that it is a measurable system attribute. That is, it is possible to specify the level of reliability that is required, monitor the system's operation over time, and check if the required reliability has been achieved. For example, a reliability requirement might be that system failures that require a reboot should not occur more than once per week. Every time such a failure occurs, it can be logged and you can check if the required level of reliability has been achieved. If not, you either modify your reliability requirement or submit a change request to address the underlying system problems. You may decide to accept a lower level of reliability because of the costs of changing the system to improve reliability or because fixing the problem may have adverse side effects, such as lower performance or throughput.

By contrast, both safety and security are about avoiding undesirable situations, rather than specifying a desired 'level' of safety or security. Even one such situation in the lifetime of a system may be unacceptable and, if it occurs, system changes have to be made. It makes no sense to make statements like 'system faults should result in fewer than 10 injuries per year.' As soon as one injury occurs, the system problem must be rectified.

Reliability requirements are, therefore, of two kinds:

1. Non-functional requirements, which define the number of failures that are acceptable during normal use of the system, or the time in which the system is unavailable for use. These are quantitative reliability requirements.
2. Functional requirements, which define system and software functions that avoid, detect, or tolerate faults in the software and so ensure that these faults do not lead to system failure.

Quantitative reliability requirements lead to related functional system requirements. To achieve some required level of reliability, the functional and design requirements of the system should specify the faults to be detected and the actions that should be taken to ensure that these faults do not lead to system failures.

The process of reliability specification can be based on the general risk-driven specification process shown in Figure 12.1:

1. *Risk identification* At this stage, you identify the types of system failures that may lead to economic losses of some kind. For example, an e-commerce system may be unavailable so that customers cannot place orders, or a failure that corrupts data may require time to restore the system database from a backup and rerun transactions that have been processed. The list of possible failure types, shown in Figure 12.6, can be used as a starting point for risk identification.
2. *Risk analysis* This involves estimating the costs and consequences of different types of software failure and selecting high-consequence failures for further analysis.

Failure type	Description
Loss of service	The system is unavailable and cannot deliver its services to users. You may separate this into loss of critical services and loss of non-critical services, where the consequences of a failure in non-critical services are less than the consequences of critical service failure.
Incorrect service delivery	The system does not deliver a service correctly to users. Again, this may be specified in terms of minor and major errors or errors in the delivery of critical and non-critical services.
System/data corruption	The failure of the system causes damage to the system itself or its data. This will usually but not necessarily be in conjunction with other types of failures.

Figure 12.6 Types of system failure

3. *Risk decomposition* At this stage, you do a root cause analysis of serious and probable system failures. However, this may be impossible at the requirements stage as the root causes may depend on system design decisions. You may have to return to this activity during design and development.

4. *Risk reduction* At this stage, you should generate quantitative reliability specifications that set out the acceptable probabilities of the different types of failures. These should, of course, take into account the costs of failures. You may use different probabilities for different system services. You may also generate functional reliability requirements. Again, this may have to wait until system design decisions have been made. However, as I discuss in Section 12.3.2, it is sometimes difficult to create quantitative specifications. You may only be able to identify functional reliability requirements.

12.3.1 Reliability metrics

In general terms, reliability can be specified as a probability that a system failure will occur when a system is in use within a specified operating environment. If you are willing to accept, for example, that 1 in any 1,000 transactions may fail, then you can specify the failure probability as 0.001. This doesn't mean, of course, that you will see 1 failure in every 1,000 transactions. It means that if you observe N thousand transactions, the number of failures that you observe should be around N. You can refine this for different kinds of failure or for different parts of the system. You may decide that critical components must have a lower probability of failure than noncritical components.

There are two important metrics that are used to specify reliability plus an additional metric that is used to specify the related system attribute of availability. The choice of metric depends on the type of system that is being specified and the requirements of the application domain. The metrics are:

1. *Probability of failure on demand (POFOD)* If you use this metric, you define the probability that a demand for service from a system will result in a system

Availability	Explanation
0.9	The system is available for 90% of the time. This means that, in a 24-hour period (1,440 minutes), the system will be unavailable for 144 minutes.
0.99	In a 24-hour period, the system is unavailable for 14.4 minutes.
0.999	The system is unavailable for 84 seconds in a 24-hour period.
0.9999	The system is unavailable for 8.4 seconds in a 24-hour period. Roughly, one minute per week.

Figure 12.7 Availability specification

failure. So, POFOD = 0.001 means that there is a 1/1,000 chance that a failure will occur when a demand is made.

2. *Rate of occurrence of failures (ROCOF)* This metric sets out the probable number of system failures that are likely to be observed relative to a certain time period (e.g., an hour), or to the number of system executions. In the example above, the ROCOF is 1/1,000. The reciprocal of ROCOF is the mean time to failure (MTTF), which is sometimes used as a reliability metric. MTTF is the average number of time units between observed system failures. Therefore, a ROCOF of two failures per hour implies that the mean time to failure is 30 minutes.

3. *Availability (AVAIL)* The availability of a system reflects its ability to deliver services when requested. AVAIL is the probability that a system will be operational when a demand is made for service. Therefore, an availability of 0.9999, means that, on average, the system will be available for 99.99% of the operating time. Figure 12.7 shows what different levels of availability mean in practice.

POFOD should be used as a reliability metric in situations where a failure on demand can lead to a serious system failure. This applies irrespective of the frequency of the demands. For example, a protection system that monitors a chemical reactor and shuts down the reaction if it is overheating should have its reliability specified using POFOD. Generally, demands on a protection system are infrequent as the system is a last line of defense, after all other recovery strategies have failed. Therefore a POFOD of 0.001 (1 failure in 1,000 demands) might seem to be risky, but if there are only two or three demands on the system in its lifetime, then you will probably never see a system failure.

ROCOF is the most appropriate metric to use in situations where demands on systems are made regularly rather than intermittently. For example, in a system that handles a large number of transactions, you may specify a ROCOF of 10 failures per day. This means that you are willing to accept that an average of 10 transactions per day will not complete successfully and will have to be canceled. Alternatively, you may specify ROCOF as the number of failures per 1,000 transactions.

If the absolute time between failures is important, you may specify the reliability as the mean time between failures. For example, if you are specifying the required

reliability for a system with long transactions (such as a computer-aided design system), you should specify the reliability with a long mean time to failure. The MTTF should be much longer than the average time that a user works on his or her models without saving their results. This would mean that users would be unlikely to lose work through a system failure in any one session.

To assess the reliability of a system, you have to capture data about its operation. The data required may include:

1. The number of system failures given a number of requests for system services. This is used to measure the POFOD.
2. The time or the number of transactions between system failures plus the total elapsed time or total number of transactions. This is used to measure ROCOF and MTTF.
3. The repair or restart time after a system failure that leads to loss of service. This is used in the measurement of availability. Availability does not just depend on the time between failures but also on the time required to get the system back into operation.

The time units that may be used are calendar time or processor time or a discrete unit such as number of transactions. In systems that spend much of their time waiting to respond to a service request, such as telephone switching systems, the time unit that should be used is processor time. If you use calendar time, then this will include the time when the system was doing nothing.

You should use calendar time for systems that are in continuous operation. Monitoring systems, such as alarm systems, and other types of process control systems fall into this category. Systems that process transactions such as bank ATMs or airline reservation systems have variable loads placed on them depending on the time of day. In these cases, the unit of 'time' used could be the number of transactions (i.e., the ROCOF would be number of failed transactions per N thousand transactions).

12.3.2 Non-functional reliability requirements

Non-functional reliability requirements are quantitative specifications of the required reliability and availability of a system, calculated using one of the metrics described in the previous section. Quantitative reliability and availability specification has been used for many years in safety-critical systems but is only rarely used in business critical systems. However, as more and more companies demand 24/7 service from their systems, it is likely that such techniques will be increasingly used.

There are several advantages in deriving quantitative reliability specifications:

1. The process of deciding what required level of the reliability helps to clarify what stakeholders really need. It helps stakeholders understand that there are different types of system failure, and it makes clear to them that high levels of reliability are very expensive to achieve.

2. It provides a basis for assessing when to stop testing a system. You stop when the system has achieved its required reliability level.
3. It is a means of assessing different design strategies intended to improve the reliability of a system. You can make a judgment about how each strategy might lead to the required levels of reliability.
4. If a regulator has to approve a system before it goes into service (e.g., all systems that are critical to flight safety on an aircraft are regulated), then evidence that a required reliability target has been met is important for system certification.

To establish the required level of system reliability, you have to consider the associated losses that could result from a system failure. These are not simply financial losses, but also loss of reputation for a business. Loss of reputation means that customers will go elsewhere. Although the short-term losses from a system failure may be relatively small, the longer-term losses may be much more significant. For example, if you try to access an e-commerce site and find that it is unavailable, you may try to find what you want elsewhere rather than wait for the system to become available. If this happens more than once, you will probably not shop at that site again.

The problem with specifying reliability using metrics such as POFOD, ROCOF, and AVAIL is that it is possible to overspecify reliability and thus incur high development and validation costs. The reason for this is that system stakeholders find it difficult to translate their practical experience into quantitative specifications. They may think that a POFOD of 0.001 (1failure in 1,000 demands) represents a relatively unreliable system. However, as I have explained, if demands for a service are uncommon, it actually represents a very high level of reliability.

If you specify reliability as a metric, it is obviously important to assess that the required level of reliability has been achieved. You do this assessment as part of system testing. To assess the reliability of a system statistically, you have to observe a number of failures. If you have, for example, a POFOD of 0.0001 (1 failure in 10,000 demands), then you may have to design tests that make 50 or 60 thousand demands on a system and where several failures are observed. It may be practically impossible to design and implement this number of tests. Therefore, overspecification of reliability leads to very high testing costs.

When you specify the availability of a system, you may have similar problems. Although a very high level of availability may seem to be desirable, most systems have very intermittent demand patterns (e.g., a business system will mostly be used during normal business hours) and a single availability figure does not really reflect user needs. You need high availability when the system is being used but not at other times. Depending, of course, on the type of system, there may be no real practical difference between an availability of 0.999 and an availability of 0.9999.

A fundamental problem with overspecification is that it may be practically impossible to show that a very high level of reliability or availability has been achieved. For example, say a system was intended for use in a safety-critical application and was therefore required to never fail over its total lifetime. Assume that 1,000 copies of the system are to be installed and the system is executed 1,000

times per second. The projected lifetime of the system is 10 years. The total number of system executions is therefore approximately $3*10^{14}$. There is no point in specifying that the rate of occurrence of failure should be $1/10^{15}$ executions (this allows for some safety factor) as you cannot test the system for long enough to validate this level of reliability.

Organizations must therefore be realistic about whether it is worth specifying and validating a very high level of reliability. High reliability levels are clearly justified in systems where reliable operation is critical, such as telephone switching systems, or where system failure may result in large economic losses. They are probably not justified for many types of business or scientific systems. Such systems have modest reliability requirements, as the costs of failure are simply processing delays and it is straightforward and relatively inexpensive to recover from these.

There are a number of steps that you can take to avoid the overspecification of system reliability:

1. Specify the availability and reliability requirements for different types of failures. There should be a lower probability of serious failures occurring than minor failures.
2. Specify the availability and reliability requirements for different services separately. Failures that affect the most critical services should be specified as less probable than those with only local effects. You may decide to limit the quantitative reliability specification to the most critical system services.
3. Decide whether you really need high reliability in a software system or whether the overall system dependability goals can be achieved in other ways. For example, you may use error detection mechanisms to check the outputs of a system and have processes in place to correct errors. There may then be no need for a high level of reliability in the system that generates the outputs.

To illustrate this latter point, consider the reliability requirements for a bank ATM system that dispenses cash and provides other services to customers. If there are hardware or software ATM problems, then these lead to incorrect entries in the customer account database. These could be avoided by specifying a very high level of hardware and software reliability in the ATM.

However, banks have many years of experience of how to identify and correct incorrect account transactions. They use accounting methods to detect when things have gone wrong. Most transactions that fail can simply be canceled, resulting in no loss to the bank and minor customer inconvenience. Banks that run ATM networks therefore accept that ATM failures may mean that a small number of transactions are incorrect but they think it more cost effective to fix these later rather than to incur very high costs in avoiding faulty transactions.

For a bank (and for the bank's customers), the availability of the ATM network is more important than whether or not individual ATM transactions fail. Lack of availability means more demand on counter services, customer dissatisfaction, engineering costs to repair the network, etc. Therefore, for transaction-based systems, such as

banking and e-commerce systems, the focus of reliability specification is usually on specifying the availability of the system.

To specify the availability of an ATM network, you should identify the system services and specify the required availability for each of these. These are:

- the customer account database service;
- the individual services provided by an ATM such as 'withdraw cash,' 'provide account information,' etc.

Here, the database service is most critical as failure of this service means that all of the ATMs in the network are out of action. Therefore, you should specify this to have a high level of availability. In this case, an acceptable figure for database availability (ignoring issues such as scheduled maintenance and upgrades) would probably be around 0.9999, between 7 am and 11 pm. This means a down time of less than one minute per week. In practice, this would mean that very few customers would be affected and would only lead to minor customer inconvenience.

For an individual ATM, the overall availability depends on mechanical reliability and the fact that it can run out of cash. Software issues are likely to have less effect than factors such as these. Therefore, a lower level of availability for the ATM software is acceptable. The overall availability of the ATM software might therefore be specified as 0.999, which means that a machine might be unavailable for between one and two minutes each day.

To illustrate failure-based reliability specification, consider the reliability requirements for the control software in the insulin pump. This system delivers insulin a number of times per day and monitors the user's blood glucose several times per hour. Because the use of the system is intermittent and failure consequences are serious, the most appropriate reliability metric is POFOD (probability of failure on demand).

There are two possible types of failure in the insulin pump:

1. Transient software failures that can be repaired by user actions such as resetting or recalibrating the machine. For these types of failures, a relatively low value of POFOD (say 0.002) may be acceptable. This means that one failure may occur in every 500 demands made on the machine. This is approximately once every 3.5 days, because the blood sugar is checked about five times per hour.
2. Permanent software failures that require the software to be reinstalled by the manufacturer. The probability of this type of failure should be much lower. Roughly once a year is the minimum figure, so POFOD should be no more than 0.00002.

However, failure to deliver insulin does not have immediate safety implications, so commercial factors rather than the safety factors govern the level of reliability required. Service costs are high because users need fast repair and replacement. It is in the manufacturer's interest to limit the number of permanent failures that require repair.

RR1: A pre-defined range for all operator inputs shall be defined and the system shall check that all operator inputs fall within this pre-defined range. (Checking)
RR2: Copies of the patient database shall be maintained on two separate servers that are not housed in the same building. (Recovery, redundancy)
RR3: N-version programming shall be used to implement the braking control system. (Redundancy)
RR4: The system must be implemented in a safe subset of Ada and checked using static analysis. (Process)

Figure 12.8 Examples of functional reliability requirements

12.3.3 Functional reliability specification

Functional reliability specification involves identifying requirements that define constraints and features that contribute to system reliability. For systems where the reliability has been quantitatively specified, these functional requirements may be necessary to ensure that a required level of reliability is achieved.

There are three types of functional reliability requirements for a system:

1. *Checking requirements* These requirements identify checks on inputs to the system to ensure that incorrect or out-of-range inputs are detected before they are processed by the system.
2. *Recovery requirements* These requirements are geared to helping the system recover after a failure has occurred. Typically, these requirements are concerned with maintaining copies of the system and its data and specifying how to restore system services after a failure.
3. *Redundancy requirements* These specify redundant features of the system that ensure that a single component failure does not lead to a complete loss of service. I discuss this in more detail in the next chapter.

In addition, the reliability requirements may include process requirements for reliability. These are requirements to ensure that good practice, known to reduce the number of faults in a system, is used in the development process. Some examples of functional reliability and process requirements are shown in Figure 12.8.

There are no simple rules for deriving functional reliability requirements. In organizations that develop critical systems, there is usually organizational knowledge about possible reliability requirements and how these impact the actual reliability of a system. These organizations may specialize in specific types of system such as railway control systems, so the reliability requirements can be reused across a range of systems.